Università degli Studi di Napoli – Federico II

Il livello microarchitetturale MIC-1

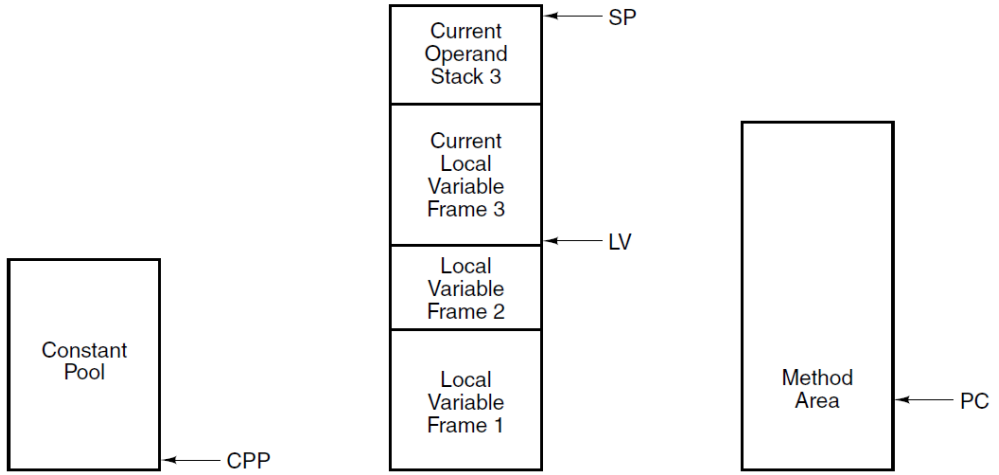
ARCHITETTURA *dei* **SISTEMI DIGITALI**

Prof.ssa Alessandra De Benedictis

IL LIVELLO MICROARCHITETTURALE

- Il livello microarchitetturale fornisce una astrazione delle unità funzionali coinvolte all'interno di una CPU e delle loro interazioni per l'implementazione dell'ISA
- Di seguito, faremo un esempio di una ISA reale, un sottoinsieme della Java Virtual Machine che opera su numeri interi (**IJVM**) e analizzeremo la microarchitettura che consente di implementarla, introdotta da Andrew Tanenbaum e denominata **MIC-1**
- Prima di introdurre la microarchitettura, diamo qualche dettaglio sul modello di memoria usato dalla IJVM e sulla sua ISA, in modo da capire in che modo operano le istruzioni, quali registri sono coinvolti e la loro funzione

IJVM – IL MODELLO DELLA MEMORIA



- La memoria contiene 4GB organizzati in locazioni (word) di 32 bit (tranne che per l'area metodi che è indirizzata a byte).
- La memoria contiene uno **stack** (LV frame + operand stack) che <u>cresce per indirizzi crescenti</u>.
- Le locazioni della memoria non possono essere accedute direttamente ma solo tramite specifici puntatori.

- La memoria è costituita da 4 aree:
  - **Constant pool**: caricata quando un programma viene caricato in memoria, contiene costanti, stringhe e puntatori e non può essere sovrascritta. La base di quest'area è puntata dal puntatore **CPP**
  - **Local variable frame**: contiene le variabili locali di un metodo ed è allocato nello stack. Alla base di quest'area (puntata dal puntatore **LV**) ci sono i parametri (arguments) con cui il metodo è stato invocato
  - **Operand stack**: separato ma attiguo al local variable frame; ha una dimensione massima che viene precalcolata dal compilatore. Il puntatore **SP** punta sempre alla cima dello stack
  - **Method area**: area programma, contiene i metodi ed è gestita come un array di byte. Il puntatore **PC** punta alla prossima istruzione da prelevare

IJVM – LO STACK E LE VARIABILI LOCALI

- Nella IJVM le variabili sono allocate nello **stack** all'interno del **local variable frame**
- Un apposito registro del datapath (**LV** - Local Variable) punta alla base dell'area allocata alla procedura correntemente attiva, mentre un altro registro **SP** - Stack Pointer punta all'ultima variabile allocata nello stack.
- Il **local variable frame** è la porzione fra LV e SP (estremi compresi)
- Le variabili vengono accedute fornendo lo **spiazzamento rispetto a LV**

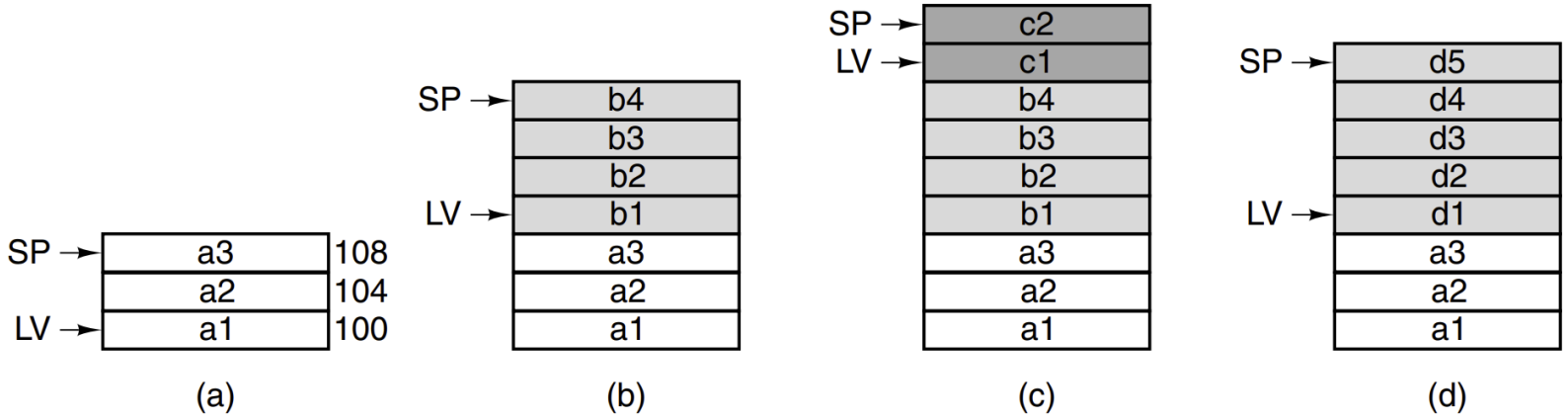


Figure 4-8. Use of a stack for storing local variables. (a) While *A* is active. (b) After *A* calls *B*. (c) After *B* calls *C*. (d) After *C* and *B* return and *A* calls *D*.

IJVM – LO STACK E GLI OPERANDI

- Lo stack viene usato anche per memorizzare gli operandi durante l'esecuzione di un'espressione aritmetica; in tal caso si parla di **operand stack**
- Supponiamo che A, dopo aver inserito sullo stack le proprie variabili locali a1, a2, a3, debba calcolare a1=a2+a3

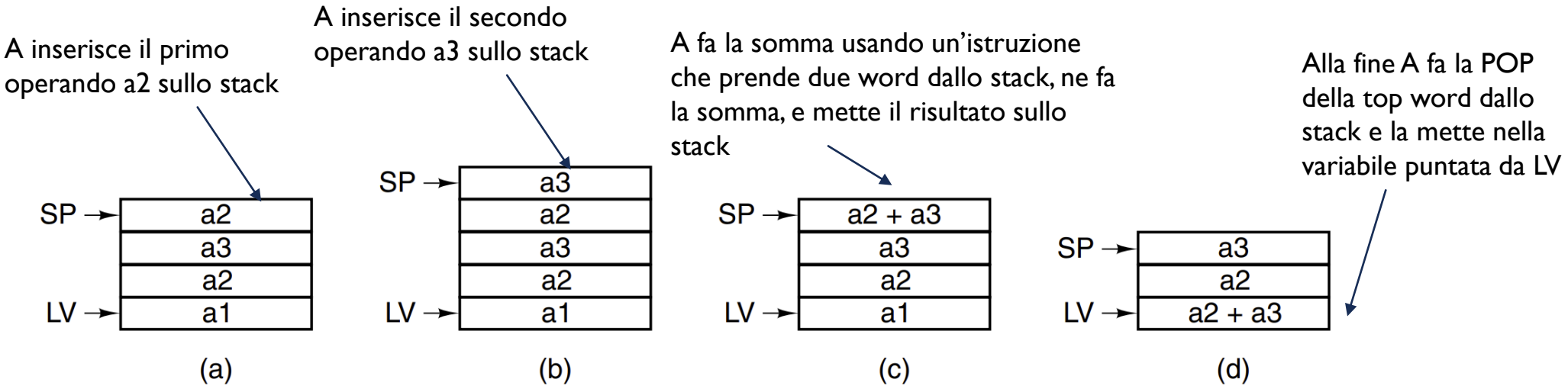


Figure 4-9. Use of an operand stack for doing an arithmetic computation.

IJVM - ISTRUZIONI

- La IJVM ha istruzioni corte con pochi operandi ed è una architettura stack-based
- Ogni istruzione ha un **opcode** e **al più un operando**, come un costante o un memory offset
- Le istruzioni consentono di:
  - Inserire una word sullo stack da varie sorgenti (constant pool-LDC_W, local variable frame – ILOAD - o istruzione stessa - BIPUSH) o prelevare una word dallo stack (ISTORE)
  - Eseguire addizione (IADD), sottrazione (ISUB), and (IAND) e or (IOR) logiche prelevando 2 operandi dallo stack e inserendo il risultato sullo stack
  - Eseguire dei salti incondizionati (GOTO) o condizionati (IFEQ,IFLT,IF_ICMPEQ)
  - Scambiare le prime due word sullo stack (SWAP), duplicare la top word (DUP) o rimuovere la top word (POP)
  - Invocare (INVOKEVIRTUAL) e ritornare (IRETURN) da una procedura
- Non sono previste istruzioni di I/O (così come nella JVM)

Hex	Mnemonic	Meaning
0x10	BIPUSH *byte*	Push byte onto stack
0x59	DUP	Copy top word on stack and push onto stack
0xA7	GOTO *offset*	Unconditional branch
0x60	IADD	Pop two words from stack; push their sum
0x7E	IAND	Pop two words from stack; push Boolean AND
0x99	IFEQ *offset*	Pop word from stack and branch if it is zero
0x9B	IFLT *offset*	Pop word from stack and branch if it is less than zero
0x9F	IF_ICMPEQ *offset*	Pop two words from stack; branch if equal
0x84	IINC *varnum const*	Add a constant to a local variable
0x15	ILOAD *varnum*	Push local variable onto stack
0xB6	INVOKEVIRTUAL *disp*	Invoke a method
0x80	IOR	Pop two words from stack; push Boolean OR
0xAC	IRETURN	Return from method with integer value
0x36	ISTORE *varnum*	Pop word from stack and store in local variable
0x64	ISUB	Pop two words from stack; push their difference
0x13	LDC_W *index*	Push constant from constant pool onto stack
0x00	NOP	Do nothing
0x57	POP	Delete word on top of stack
0x5F	SWAP	Swap the two top words on the stack
0xC4	WIDE	Prefix instruction; next instruction has a 16-bit index

IJVM – ISTRUZIONI – CHIAMATA A METODO (1/4)

- La IJVM semplifica il meccanismo di chiamata a procedura di Java in cui è possibile invocare metodi su altri oggetti, stabilendo che si possano richiamare solo metodi definiti nello stesso oggetto

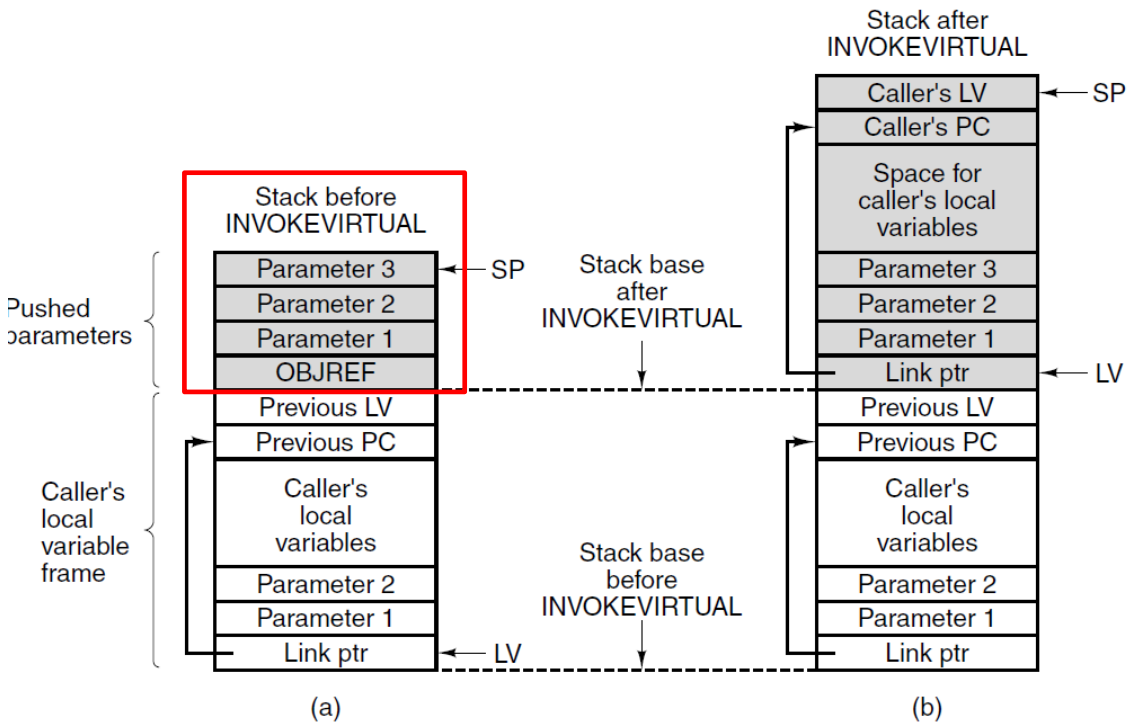


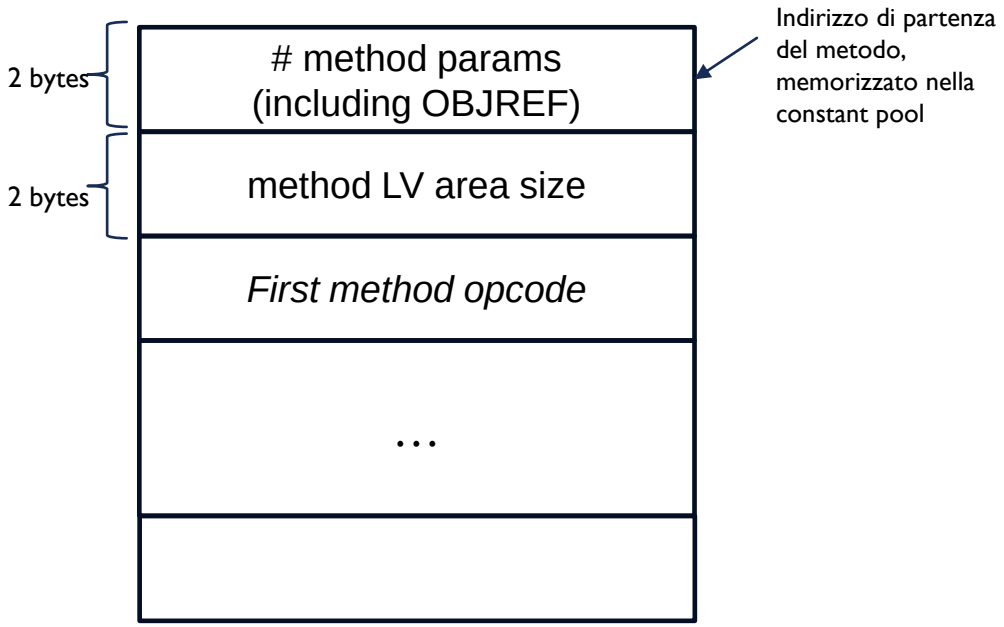
Figure 4-12. (a) Memory before executing INVOKEVIRTUAL. (b) After executing it.

- Il chiamante:

1. mette sullo stack un puntatore all'oggetto da chiamare (OBJREF) (non serve nella IJVM ma viene inserito per consistenza rispetto alla JVM)
2. mette nello stack i parametri della procedura
3. chiama l'istruzione INVOKEVIRTUAL

INVOKEVIRTUAL ha come operando la posizione (intesa come *offset* rispetto a CPP) nella *constant pool* dell'indirizzo di partenza della procedura da chiamare (che è definita nella Method Area)

IJVM – ISTRUZIONI – CHIAMATA A METODO (2/4)



Nella method area all'indirizzo corrispondente alla procedura si trovano 4 byte «speciali»:

- I primi 2 byte contengono il numero (codificato come intero su 16 bit) di parametri del metodo (il conteggio include OBJREF) => con questo valore è possibile accedere a OBJREF partendo da SP
- I successivi 2 byte contengono la size della local variable area del metodo (parametri+variabili locali) => è fondamentale perché bisognerà allocare dello spazio sullo stack per il metodo

IJVM – ISTRUZIONI – CHIAMATA A METODO (3/4)

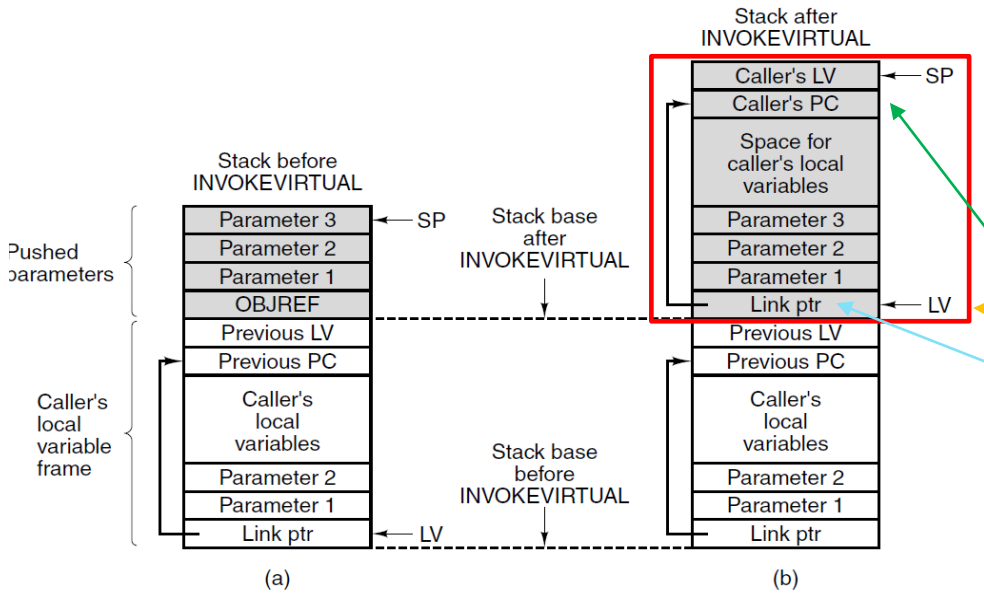
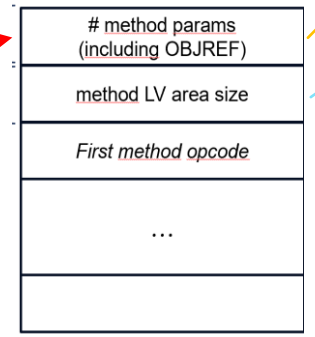


Figure 4-12. (a) Memory before executing INVOKEVIRTUAL. (b) After executing it.

Indirizzo base del metodo nella **method area**



- L'istruzione INVOKEVIRTUAL <offset>:
  1. Utilizza l'offset indicato nell'istruzione per accedere all'indirizzo di partenza del metodo nella Constant Pool
  2. Accede all'indirizzo base del metodo nella Method Area e usa i 16 bit che indicano il numero di parametri del metodo per calcolare la base del nuovo LV frame **(LV = SP-#params)** => (LV punterà a OBJREF)
  3. Calcola l'indirizzo nello stack dove dovrà salvare il vecchio PC (quello del chiamante) detto **link pointer (Link_ptr = LV + *method LV area size*)**. Inserisce questo indirizzo nella locazione puntata da LV
  4. Salva sullo stack il vecchio PC (nell'indirizzo appena calcolato) e il vecchio LV (SP punterà al vecchio LV)
  5. Inserisce in PC l'indirizzo della prima istruzione utile del metodo

IJVM – ISTRUZIONI – CHIAMATA A METODO (4/4)

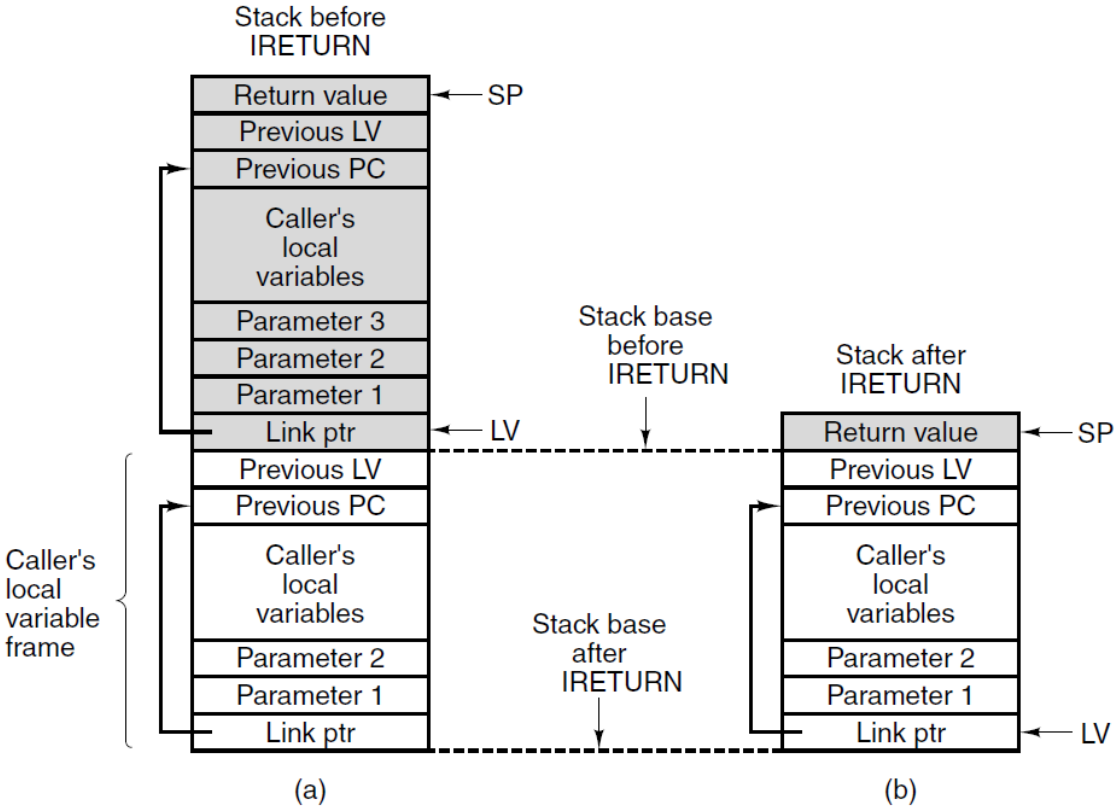
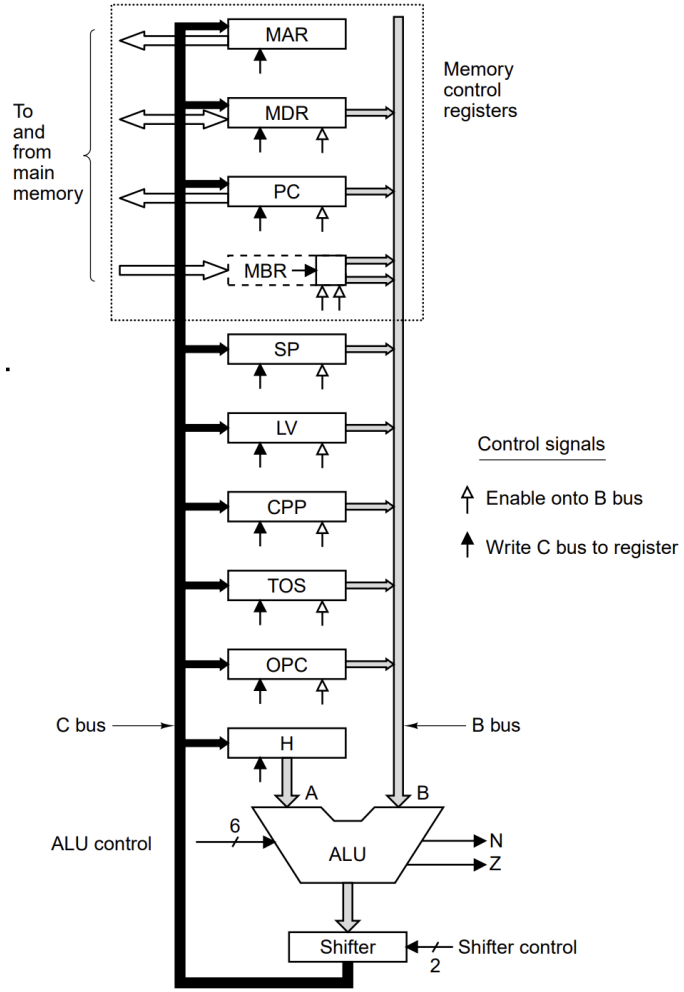


Figure 4-13. (a) Memory before executing IRETURN. (b) After executing it.

L'istruzione IRETURN:

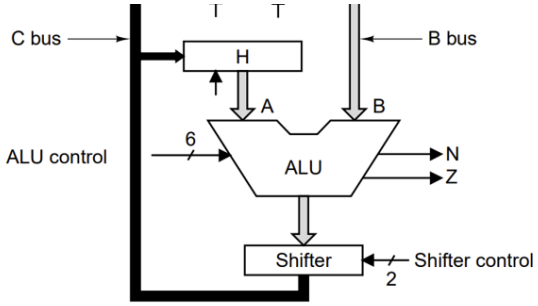
1. Accede al link pointer e ottiene l'indirizzo sullo stack del vecchio PC da ripristinare e del vecchio LV (la word successiva)
2. Copia il valore di ritorno che si trova in cima allo stack nella locazione puntata da LV, e quindi anche SP punterà a questa locazione
3. Ripristina i valori di PC e LV

MICROARCHITETTURA MIC-1 – DATA PATH



- Ogni istruzione della IJVM viene implementata attraverso una sequenza di microistruzioni gestite secondo l'approccio microprogrammato
- Il data path del MIC-1, che consente di implementare le istruzioni IJVM, è costituito da una ALU, da una serie di registri a 32 bit accessibili solo a livello microarchitetturale, e da un BUS B per la lettura dai registri e un BUS C per la scrittura sui registri

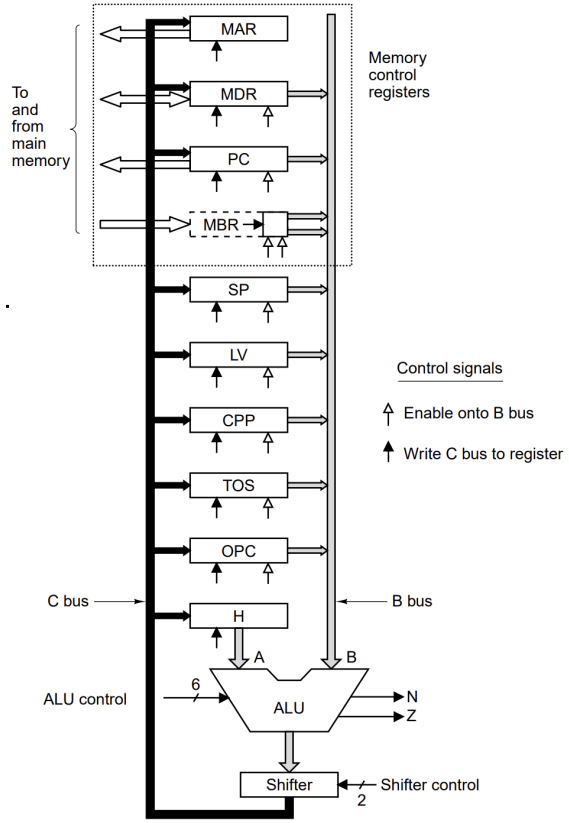
MICROARCHITETTURA MIC-1 – DATA PATH - ALU

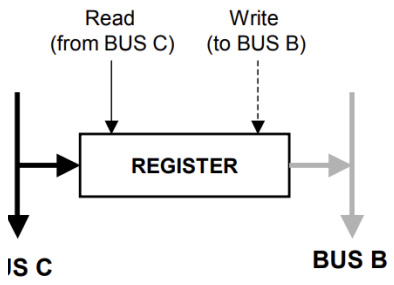


F_0	F_1	ENA	ENB	INVA	INC	Function
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\overline{A}$
1	0	1	1	0	0	$\overline{B}$
1	1	1	1	0	0	A + B
1	1	1	1	0	1	A + B + 1
1	1	1	0	0	1	A + 1
1	1	0	1	0	1	B + 1
1	1	1	1	1	1	B – A
1	1	0	1	1	1	B – 1
1	1	1	0	1	1	–A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
0	1	0	0	0	1	1
0	1	0	0	1	0	–1

- La ALU ha 6 linee di controllo (teoricamente ci sono 64 combinazioni possibili, ma non tutte sono utilizzate):
  - F0, F1 per l'operazione;
  - ENA, ENAB per abilitare gli inputs A e B;
  - INVA per invertire l'ingresso A (left registro H);
  - INC per incrementare di 1;
- La ALU ha 2 segnali di uscita, N e Z che segnalano che l'operazione effettuata ha avuto un risultato negativo o nullo, rispettivamente
- Quando la ALU deve fare operazioni su 2 operandi:
  - si carica il primo operando prendendolo da uno dei registri, mettendolo sul bus B e selezionando la configurazione di controllo che pone in uscita B; il valore viene poi copiato dal bus C nel registro H.
  - Si connette il secondo operando direttamente col bus B
- Lo shift register ha altre due linee di controllo:
  - Shift left logical (SLL8): shifta il valore in input a sinistra di un byte, e riempie il byte meno significativo con tutti zero,
  - Shift Right arithmetic (SRA1): shifta il valore in input di un bit a destra e lascia invariato il bit più significativo

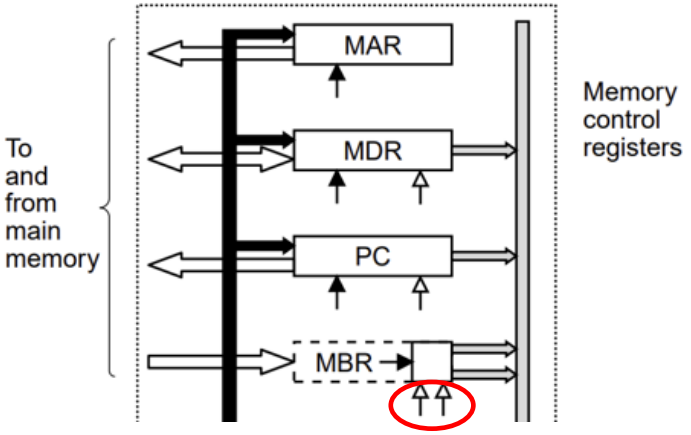
MICROARCHITETTURA MIC-1 – DATA PATH - REGISTRI



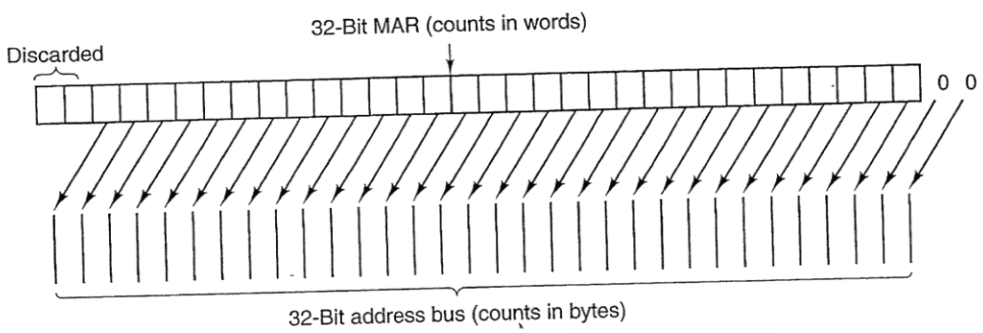


- Quasi tutti i registri dispongono di segnali di controllo per la lettura dal bus C e la scrittura sul bus B; alcuni registri hanno dei segnali di controllo aggiuntivi
- 9 registri possono scrivere su B (tutti tranne MAR) e 9 registri possono essere caricati da C (tutti tranne MBR)
- **CPP, LV, SP**: usati per la gestione della memoria
- **PC**: program counter
- **MAR** (memory address register): indirizzo della word da leggere/scrivere
- **MDR** (memory data register): word letta/da scrivere in memoria
- **TOS**: top of stack (valore puntato da SP)
- **OPC**: registro temporaneo usato dalle istruzioni
- **MBR** (memory byte register): memorizza i byte dell'istruzione man mano che viene interpretata

MICROARCHITETTURA MIC-1 – DATA PATH – COMUNICAZIONE CON LA MEMORIA



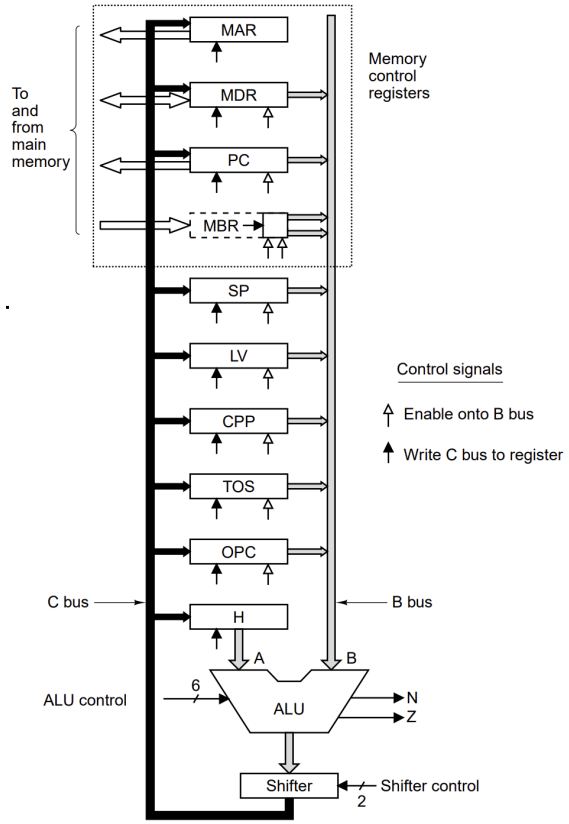
- Il MIC-1 ha due modi diversi per comunicare con la memoria:
  - 1 porto a 32 bit (indirizzabile a word) controllato da MAR/MDR;
  - 1 porto ad 8 bit (indirizzabile a byte) controllato da PC/MBR:
- MAR e PC indirizzano due parti differenti della memoria:
  - MAR indirizza word di dati, mentre PC indirizza byte di programma a livello ISA
  - PC indirizza il singolo byte, che viene letto dalla memoria e posto in MBR
  - MBR può essere posto sul bus B in due modi: unsigned o signed (con estensione del segno) a seconda di quale dei due segnali di controllo è asserito



NOTA: nell'implementazione fisica la memoria è indirizzata a byte, quindi il contenuto di MAR deve essere «aggiustato»

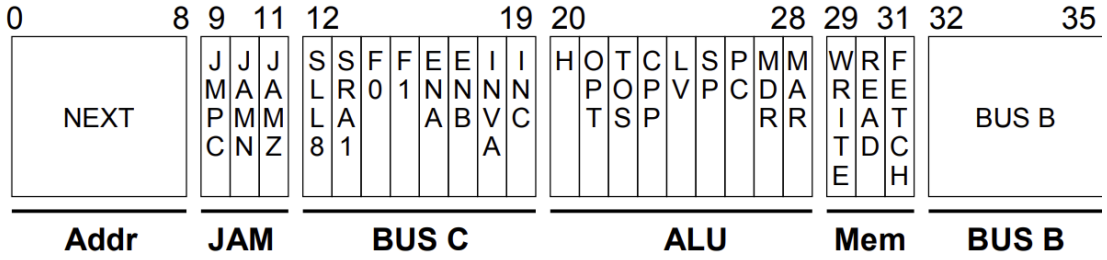
- I due bit meno significativi dell'address bus vengono messi a 0
- I due bit più significativi di MAR vengono ignorati
- Il primo bit di MAR viene connesso al terzo bit dell'address bus

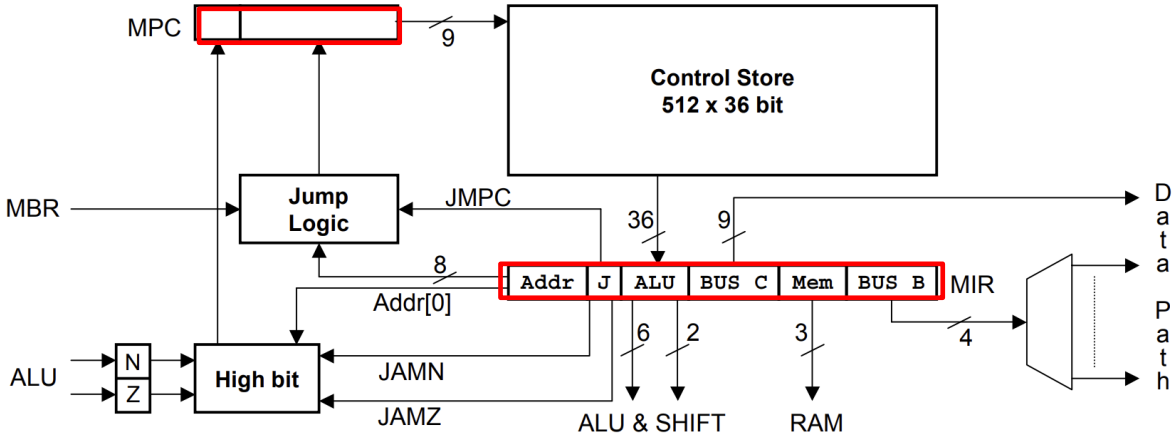
MICROARCHITETTURA MIC-1 – SEGNALI DI CONTROLLO



- Per controllare il dataPath del Mic-1 occorrono 29 segnali raggruppati in 5 tipologie:
  - 9 segnali per scrivere dal bus C ai registri;
  - 9 segnali per scrivere dal registri al bus B;
  - 8 segnali per la ALU e lo Shift register;
  - 2 segnali per indicare il read/write per MAR/MDR;
  - 1 segnale per indicare il memory fetch PC/MBR;
- Alcuni segnali sono mutuamente esclusivi (un solo registro alla volta può scrivere su B) quindi è possibile ridurre i segnali a **24**
- Sappiamo poi che per avere un controllo microprogrammato è opportuno avere all'interno della control word l'indirizzo alla prossima microistruzione da eseguire, che sarà eventualmente modificato da logica esterna per consentire salti; la IJVM allora ha 2 segnali ulteriori di controllo:
  - NEXT: Indirizzo della microistruzione seguente
  - JAM: Codici di condizione per i salti

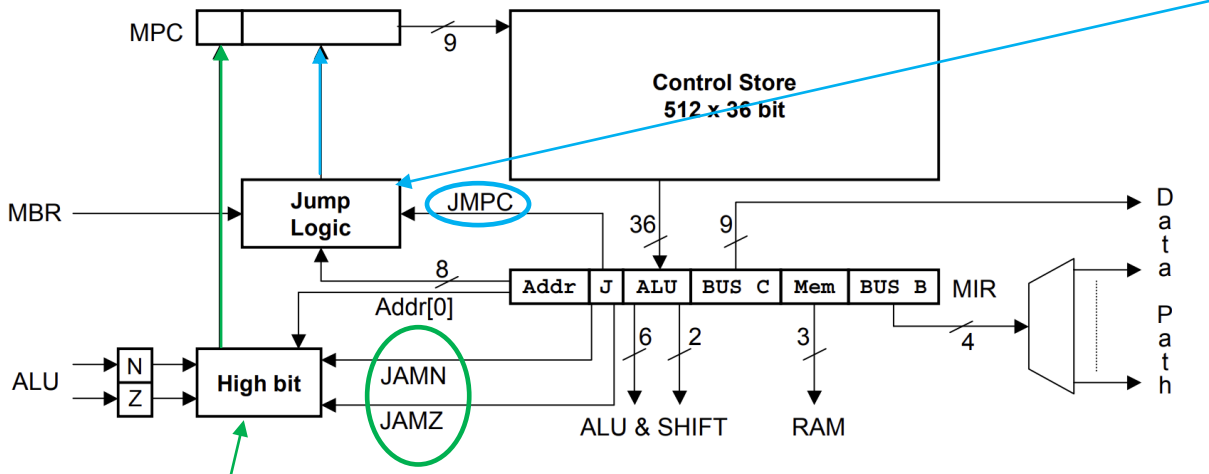
MICROARCHITETTURA MIC-1 – CONTROL STORE E CONTROL WORD





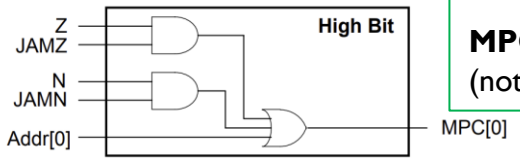
- Ogni istruzione della IJVM viene realizzata mediante l'esecuzione di una sequenza di **microistruzioni di 36 bit** ciascuna
- Le microistruzioni sono memorizzate in una **control store** formata da 512 parole di 36 bit
  - Esse non vengono eseguite nell'ordine in cui sono memorizzate
- La control store utilizza:
  - **MPC**: Micro Program Counter: indica, ad ogni passo di controllo, quale microistruzione leggere (è un registro virtuale)
  - **MIR**: Micro Instruction Register. Contiene la microistruzione letta all'indirizzo puntato da MPC
- L'**indirizzo** nella control store **della prima microistruzione** di una istruzione ISA corrisponde al **valore binario dell'opcode** dell' istruzione stessa
  - IADD opcode=0x60 => la prima microistruzione di IADD è memorizzata all'indirizzo 0x60

MICROARCHITETTURA MIC-1 – LOGICA DI SALTO



Determina il valore dei bit MPC[1..8] del micro program counter:
se JMPC=0 allora MPC[1..8]=Addr[1..8];
se JMPC=1 allora MPC[1..8]=Addr[1..8] OR MBR[0..7];

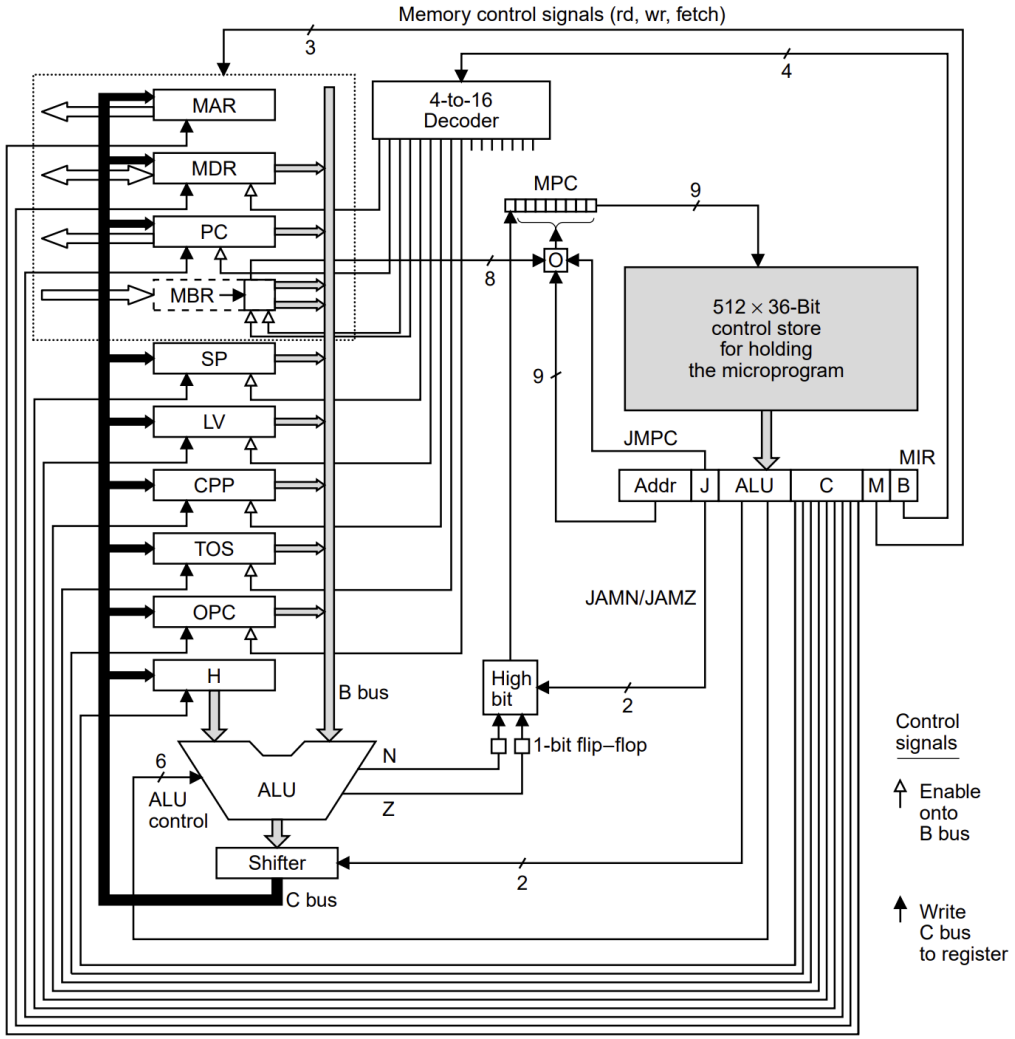
JMPC consente di implementare in maniera efficiente un multiway-branch: quando JMPC=1 gli 8 bit meno significativi di next_address sono normalmente bassi, quindi in MPC va il valore di MBR con eventualmente un 1 in testa
Tipicamene in MBR ci sarà un codice operativo, quindi ponendo JMPC=1 di fatto in MPC si va a inserire il valore dell'opcode che corrisponde all'indirizzo della prima microistruzione da eseguire per l'istruzione in esame



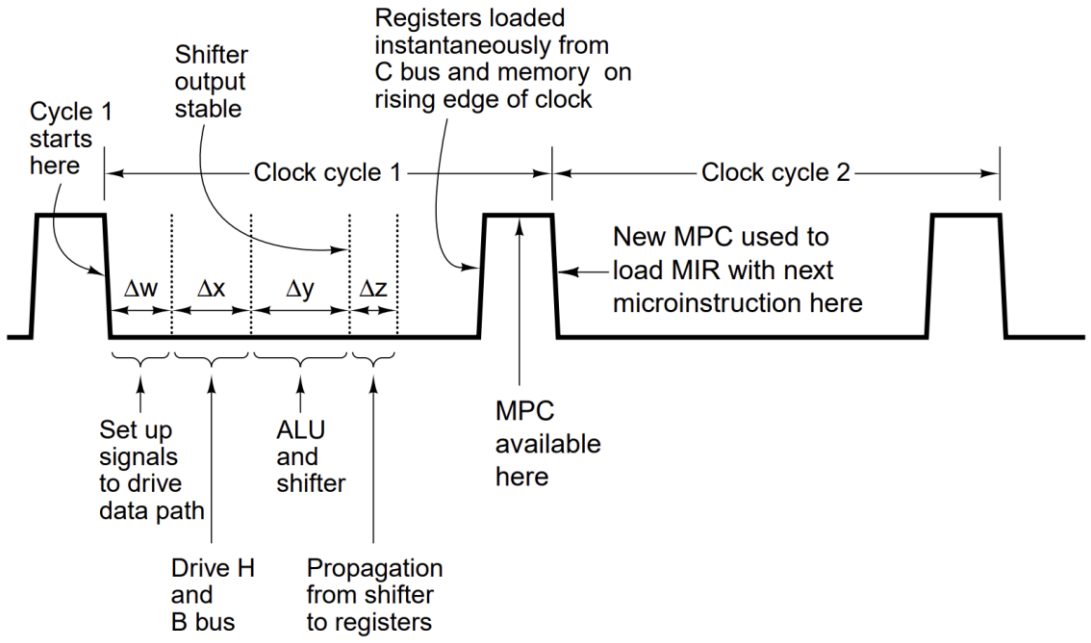
Determina il bit MPC[0] in base ai valori JAMN e JAMZ:
se JAMN=0 e JAMZ=0 allora MBR[0]=Addr[0]; (bit più significativo)
se JAMN=0 e JAMZ=1 allora MBR[0]=Addr[0] + Z;
se JAMN=1 e JAMZ=0 allora MBR[0]=Addr[0] + N;
se JAMN=1 e JAMZ=1 allora MBR[0]=Addr[0] + N + Z;

MPC[0] = Addr[0] + Z · JAMZ + N· JAMN
(nota: rispetto a next_address al più cambierebbe il bit più significativo)

MICROARCHITETTURA MIC-1 – SISTEMA COMPLETO



MICROARCHITETTURA MIC-1 – TEMPIFICAZIONE



- La tempificazione del datapath si basa su un clock asimmetrico caratterizzato da un breve impulso
- Ogni «data path cycle» parte dal **fronte di discesa** ed è suddiviso in 4 sotto-cicli (determinati dai ritardi di propagazione):
  - Δw: il registro MIR viene caricato (in base all'indirizzo nel MPC) e i segnali di controllo si stabilizzano
  - Δx: i segnali di controllo vengono propagati e il bus B viene caricato col valore contenuto nel registro selezionato
  - Δy: la ALU e lo shifter elaborano e il risultato si stabilizza
  - Δz: I segnali sul bus C e sul bus verso la memoria si stabilizzano
- Sul **fronte di salita** dell'impulso avviene il **caricamento dei registri** dal bus C, dei flip-flop N e Z, e di MBR e MDR (relativamente all'operazione eventualmente iniziata alla fine del data path cycle precedente)
  - Appena MBR è disponibile viene caricato anche MPC

MICROCODICE

- L'esecuzione di ogni istruzione ISA consiste in:
  - Lettura dell'istruzione dalla memoria
  - Salto al microcodice dell'istruzione ISA
  - Esecuzione del microcodice
  - Ritorno al **microcodice per il caricamento della prossima istruzione**
- I segnale Read e Fetch hanno significati differenti:
  - Read viene usato per leggere una word dalla memoria in MDR
  - Fetch viene usato per leggere un byte dalla memoria in MBR • Microcodice per il caricamento dell'istruzione:

Label	Microcode
MAIN1	PC=PC+1; Fetch; goto(MBR)

Questa notazione appartiene al linguaggio simbolico Micro Assembly Language (MAL)

MICROCODICE – LINGUAGGIO MAL

DEST = H
DEST = SOURCE
DEST = $\overline{H}$
DEST = $\overline{SOURCE}$
DEST = H + SOURCE
DEST = H + SOURCE + 1
DEST = H + 1
DEST = SOURCE + 1
DEST = SOURCE – H
DEST = SOURCE – 1
DEST = –H
DEST = H AND SOURCE
DEST = H OR SOURCE
DEST = 0
DEST = 1
DEST = –1

- Bisogna fare attenzione ad utilizzare solo operazioni «legali»:
- Source può essere uno fra MDR, OC, MBR, MBRU, SO, LV, CPP, TOS, OPC (questi registri possono anche essere sorgenti per la ALU sul bus B)
- Dest può essere uno fra MAP, MDR, PC, SP, LV, CPP, TOS, OPC, H (questi registri sono anche possibili destinazioni per il risultato della ALU veicolato sul bus C)
- Es. l'operazione MDR=SP+MDR non è legale perché non può essere eseguita in un solo ciclo (uno degli operandi dell'ALU deve essere prima messo in H)

MICROCODICE: ESEMPIO - IADD

- La IADD:
  - somma i 2 valori sulla cima dello stack
  - rimuove i due valori dalla cima dello stack
  - pone il risultao sulla cima dello stack

Calcola SP-1 (c'è una funzione apposta nella ALU che calcola B-1) e carica il risultato sia in MAR sia in SP; avvia una lettura da memoria (la lettura inizierà solo alla fine del ciclo, quando MAR viene caricato sul fronte di salita e il dato sarà pronto solo alla fine del prossimo ciclo, quando MDR viene caricato)

Il valore in cima allo stack viene posto in H

Il valore in MDR viene sommato con H e il risultato salvato sia in TOS sia in MDR; poi si avvia una scrittura (inizia solo alla fine del ciclo)

Label	Microcode
IADD1	MAR=SP=SP-1; Read
IADD2	H=TOS
IADD3	MDR=TOS=MDR+H; Write; goto MAIN1